CSE 4310 – Compiler Design

CH2 – Lexical Analysis

Outline

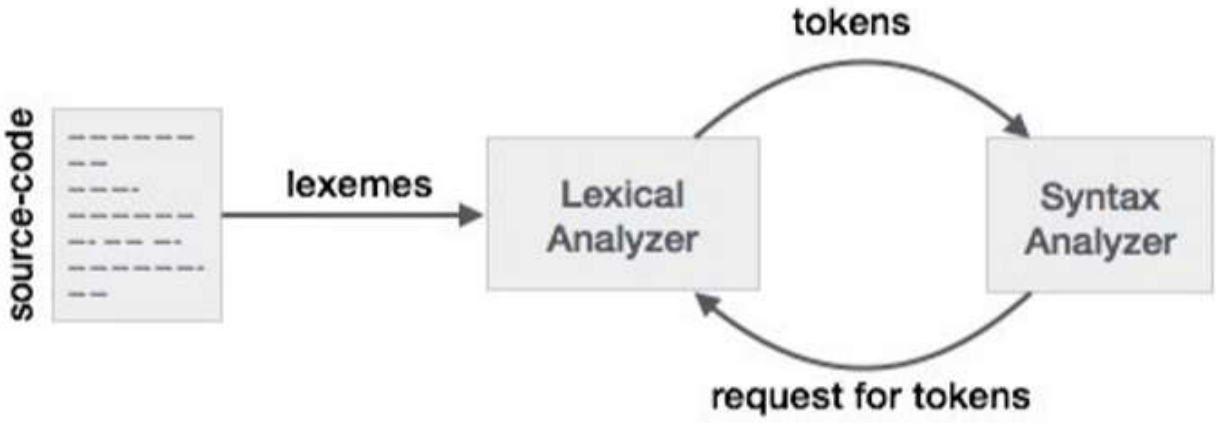
- Introduction – What is Lexical Analysis?
- Tokens, Patterns, Lexemes
- The Role of Lexical Analyzer
- Lexical Analyzer versus Parsing
- Specification of Tokens
  - Regular Expressions and Regular Definitions
- Recognition of Tokens
  - Finite Automata and Transition Diagrams
- Lex – a lexical analyzer generator

Introduction

- Lexical analysis is the first phase of a compiler.
- In this chapter you will learn about the role of Lexical Analyzer (***LA***) :
  - by understanding the terms ***tokens***, ***patterns***, ***lexemes***, ***attributes of tokens*** and ***lexical errors***.
  - the **specification of tokens** by understanding ***strings and languages***, ***operations on languages***, ***regular expressions***, and ***regular definitions***.
  - the ***generation (recognition) of tokens*** using ***finite automaton***, ***transition diagrams*** – recognition of ***reserved words***, ***identifiers***, etc.
- You will also learn about ***Lex*** – a lexical analyzer generator tool.

What is Lexical Analysis?

- The role of the lexical analyzer is to read a sequence of characters from the source program and produce ***tokens*** to be used by the parser.
  - The lexical analysis phase of the compiler is often called **tokenization**
- The ***lexical analyzer (LA)*** reads the stream of characters making up the source program and groups the characters into meaningful sequences called ***lexemes***.



What is Lexical Analysis?

- For each lexeme,
  - the lexical analyzer produces as output a ***token*** of the form ***<token–name, attribute–value>*** that it passes on to the subsequent phase, ***syntax analysis***.
- Where,
  - ***token–name*** is an abstract symbol that is used during syntax analysis , and
  - ***attribute-value*** points to an entry in the ***symbol table*** for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

What is Lexical Analysis?

Example:

- For example, suppose a source program contains the assignment statement

  `position = initial + rate * 60`

- The characters in this assignment could be grouped into the following **lexemes** and mapped into the following ***tokens*** passed on to the syntax analyzer:

  1. `position` is a lexeme that would be mapped into a token `<id, 1>`, where `id` is an abstract symbol standing for ***identifier*** and ***1*** points to the symbol table entry for `position`.
     - The symbol-table entry for an identifier holds information about the identifier, such as its ***name*** and ***type***.

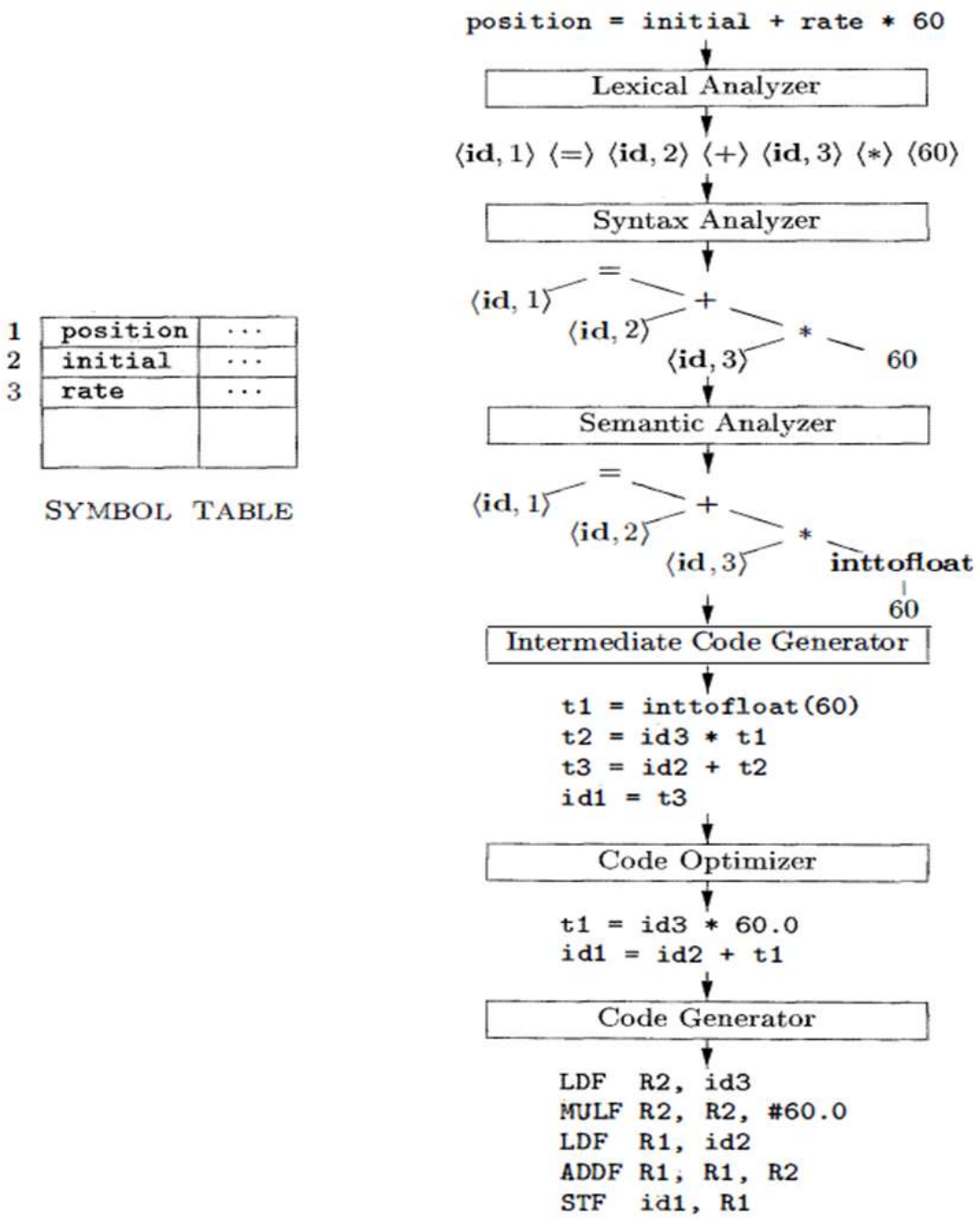
  2. The assignment symbol `=` is a lexeme that is mapped into the token `<=, >`.
     - Since this token needs no attribute-value, the second component is omitted.

What is Lexical Analysis?

Example:

3. `initial` is a lexeme that is mapped into the token `<id, 2>`, where `2` points to the symbol-table entry for initial.
4. `+` is a lexeme that is mapped into the token `<+, >`.
5. `rate` is a lexeme that is mapped into the token `<id, 3>`, where `3` points to the symbol-table entry for rate.
6. `*` is a lexeme that is mapped into the token `<*, >`.
7. `60` is a lexeme that is mapped into the token `<60, >`.

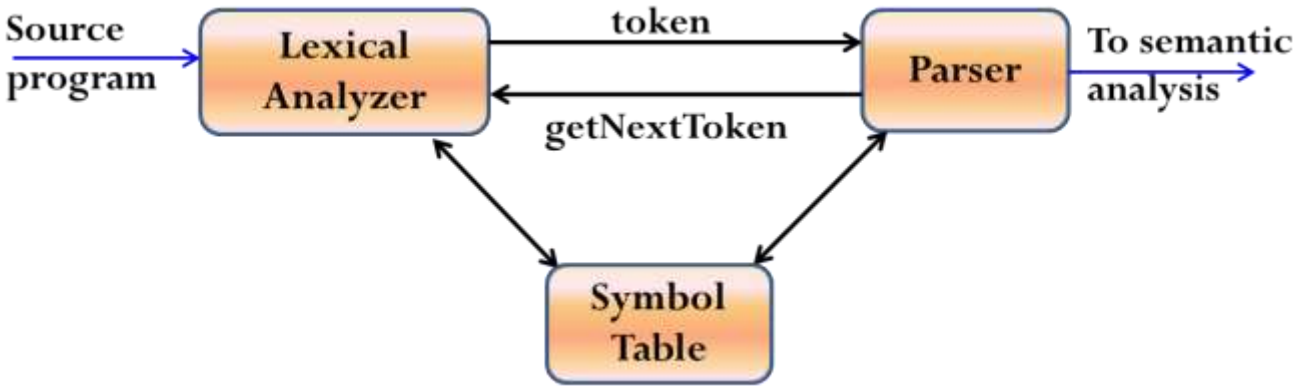
- Blanks separating the lexemes would be discarded by the lexical analyzer.



1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE

The Role of Lexical Analyzer



- It is common for the lexical analyzer to interact with the ***symbol table***.
  - When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.
  - In some cases, information regarding the kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser.

Other tasks of an LA

- Since LA is the part of the compiler that reads the source text, it may perform certain other tasks besides identification of lexemes such as:
  1. Stripping out ***comments*** and ***whitespace*** (***blank***, ***newline***, ***tab***, and perhaps other characters that are used to ***separate tokens*** in the input).
  2. ***Correlating error messages*** generated by the compiler with the source program.
     - For instance, the LA may keep track of the number of newline characters seen, so it can associate a line number with each error message.
  3. If the source program uses a ***macro-preprocessor***, the expansion of macros may also be performed by the lexical analyzer.
- Sometimes, lexical analyzers are divided into a cascade of two processes:
  A. **Scanning** consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.
  B. **Lexical analysis** is the more complex portion, where the scanner produces the sequence of tokens as output (***tokenization***)

Lexical Analysis versus Parsing

- There are a number of reasons why the analysis portion of a compiler is normally separated into lexical analysis and parsing (syntax analysis) phases:

1. **Simplicity of Design**:- is the most important consideration that often allows us to simplify compilation tasks.
   - For example, a parser that had to deal with comments and whitespace as syntactic units would be considerably more complex than one that can assume comments and whitespace have already been removed by the lexical analyzer.
   - If we are designing a new language, separating lexical and syntactic concerns can lead to a cleaner overall language design.
2. **Compiler efficiency is improved**:- a separate lexical analyzer allows us to apply specialized techniques that serve only the lexical task, not the job of parsing.
   - In addition, specialized buffering techniques for reading input characters can speed up the compiler significantly.
3. **Compiler portability is enhanced**:- input-device-specific peculiarities can be restricted to the lexical analyzer.

Tokens, Patterns, and Lexemes

- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token.
  - **Example**: `-5, i, sum, 100, for ; int 10 + -`
- **Token**:- is a pair consisting of a token name and an optional attribute value.
  - **Example**:

```
Identifiers = { i, sum }
int_Constatnt = { 10, 100, -5 }
Opr = { +, -}
rev_Words = { for, int}
Separators = { ;, ,}
```

  - One token for each **keyword**. The pattern for a keyword is the same as the keyword itself.
  - Tokens for the **operators** , either individually or in classes such as the token in comparison operators
  - One token representing all **identifiers**.
  - One or more tokens representing **constants**, such as numbers and literal strings .
  - Tokens for each **punctuation symbol**, such as left and right parentheses, comma, and semicolon

Tokens, Patterns, and Lexemes

- **Pattern** is a rule describing the set of lexemes that can represent a particular token in source program
  - It is a description of the form that the lexemes of a token may take.
  - In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword.
  - For identifiers and some other tokens, the pattern is a more complex structure that is matched by many strings.
- One efficient but complex ***brute-force approach*** is to read character by character to check if each one follows the right sequence of a token.

Tokens, Patterns, and Lexemes

- The below example shows a partial code for this brute-force lexical analyzer.
- What we'd like is a set of tools that will allow us to easily create and modify a lexical analyzer that has the same run-time efficiency as the brute-force method.
- The first tool that we use to attack this problem is Deterministic Finite Automata (DFA).

```
if (c = nextchar() == 'c') {
    if (c = nextchar() == 'l') {
       /* code to handle the rest of either "class" or any identifier that
          starts with "cl" */
    } else if (c == 'a') {
        /* code to handle the rest of either "case" or any identifier that
           starts with "ca" */
    } else {
         /* code to handle any identifier that starts with c */
    }
} else if ( c = …..) {
     // many more nested ifs for the rest of the lexical analyzer
```

Tokens, Patterns, and Lexemes

LEXEME			PATTERN			TOKEN		
A1, Sum, Total			Starting with a letter & followed by letter or digit but not a keyword			ID		
If	Then	Else	If	Then	Else	IF	THEN	ELSE
123, 123.45, 12e+07			Starting with digit followed by a digit or optional fraction and or optional exponent			NUM		
<, >, <=, >=, <>, =			< or > or <= or >= or <> or =			RELOP		
"lateral String"			Any sequence of character within Quotations but no quotation symbol included			"LITERAL-STRING"		
Punctuation Symbol , ; ( ) { }			, ; ( ) { }			, : ( ) { }		

Consideration for a simple design of Lexical Analyzer

- Lexical Analyzer can allow source program to be
  1. **Free-Format Input**:- the alignment of lexeme should not be necessary in determining the correctness of the source program, such restriction put extra load on Lexical Analyzer
  2. **Blanks Significance**:- Simplify the task of identifying tokens
     - E.g. `int a` indicates **<int is keyword> <a is identifier>**

       `inta` indicates **<inta is Identifier>**
  3. **Keyword must be reserved**:- Keywords should be reserved otherwise LA will have to predict whether the given lexeme should be treated as a ***keyword*** or as ***identifier***
     - E.g.
```
if then then then = else;
else else = then;
```
     - The above statements are misleading as ***then*** and ***else*** keywords are not reserved.

- **Approaches to implementation**
  - Use **assembly language-** Most efficient but most difficult to implement
  - Use **high level languages** like C- Efficient but difficult to implement
  - Use **tools** like ***Lex***, ***flex***, ***javacc***… Easy to implement but not as efficient as the first two cases

Lexical Errors

- Are primarily of two kinds:
  1. Lexemes whose ***length exceed the bound*** specified by the language.
     - Most languages have bound on the precision of numeric constants.
     - A constant whose bound exceeds this bound is a lexical error.
  2. ***Illegal character*** in the program
     - Characters like ~, ∀, ©, ® occurring in a given programming language (but not within a string or comment) are lexical errors.
  3. ***Unterminated*** strings or comments.

Handling Lexical Errors

- It is hard for a LA to tell, without the aid of other components, that there is a source-code error.
  - For instance, if the string `fi` is encountered for the first time in a java program in the context: `fi(a == 2*4+3)` a lexical analyzer cannot tell whether `fi` is a misspelling of the **keyword** `if` or an **undeclared function identifier**.
- However, suppose a situation arises in which the lexical analyzer is **unable to proceed** because none of the patterns for tokens matches any prefix of the remaining input.
  - The simplest recovery strategy is "***panic mode***" recovery.
    - We delete successive characters from the remaining input, until the lexical analyzer can find a well-formed token at the beginning of what input is left.
  - This recovery technique may confuse the parser, but in an interactive computing environment it may be quite adequate.

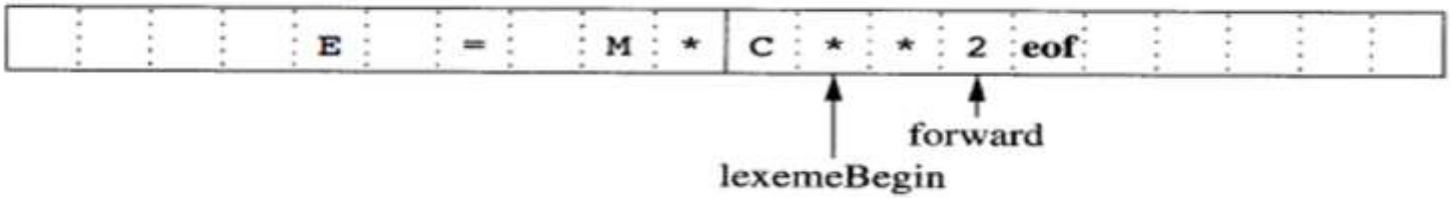
Handling Lexical Errors

- Other possible error-recovery actions are:
  1. Insert a missing character into the remaining input.
  2. Replacing an incorrect character by a correct character
  3. Transpose two adjacent characters (such as , fi => if).
  4. Deleting an extraneous character
  5. Pre-scanning

Input Buffering

- There are some ways that the task of reading the source program can be speed up
- This task is made difficult by the fact that we often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme
  - In **C language**: we need to look after **−**, **=** or **<** to decide what token to return
  - We shall introduce a ***two-buffer scheme*** that handles large ***look-aheads*** safely
- We then consider an improvement involving ***sentinels*** that saves time checking for the ends of buffers
- Because of the amount of time taken to process characters and the large number of characters that must be processed during compilation of a large source program,
  - specialized ***buffering techniques*** have been developed to reduce the amount of overhead to process a single input character
- An important scheme involves two buffers that are alternatively reloaded, as shown in the Figure next slide

Input Buffering



- Each buffer is of the same size N, and N is usually the size of a disk block, e.g., 4096 bytes
- Using one system read command we can read N characters into a buffer, rather than using one system call per character
  - If fewer than N characters remain in the input file, then a special character, represented by ***eof***, marks the end of the source file and is different from any possible character of the source program
- Two pointers to the input are maintained:
  - **Pointer *lexemeBegin***, marks the beginning of the current lexeme, whose extent we are attempting to determine
  - **Pointer *forward*** scans ahead until a pattern match is found

Input Buffering

- Once the lexeme is determined, ***forward*** is set to the character at its right end (involves retracting)
  - Then, after the lexeme is recorded as an attribute value of the token returned to the parser, ***lexemeBegin*** is set to the character immediately after the lexeme just found
- Advancing forward requires that we first test whether we have reached the end of one of the buffers, and if so, we must reload the other buffer from the input, and move forward to the beginning of the newly loaded buffer

Sentinels

- If we use the previous scheme, we must check each time we advance forward, that we have not moved off one of the buffers; if we do, then we must also reload the other buffer
  - Thus, for each character read, we must make two tests: one for the end of the buffer, and one to determine which character is read
  - We can combine the buffer-end test with the test for the current character if we extend each buffer to hold sentinel character at the end

Input Buffering

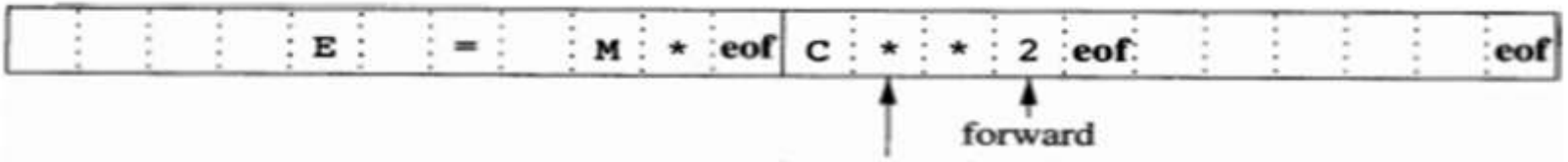


Fig. Sentinels at the end of each buffer

- The ***sentinel*** is a special character that cannot be part of the source program, and a natural choice is the character ***eof***
- Note that ***eof*** retains its use as a marker for the end of the entire input
- Any ***eof*** that appears other than at the end of buffer means the input is at an end
- The above Figure shows the same arrangement as Figure in slide no. 21, but with the sentinels added.

```
Switch (*forward++) {
    case eof:
      if (forward is at end of first buffer) {
            reload second buffer;
            forward = beginning of second buffer;
        }
      else if {forward is at end of second buffer) {
            reload first buffer;
            forward = beginning of first buffer;
        }
      else /*eof within buffer marks the end of input */
            terminate lexical analysis;
            break;
            cases for the other characters;
}
```

Specification of Tokens

- ***Regular expressions*** are an important notation for specifying lexeme patterns.
- While they cannot express all possible patterns, they are very effective in specifying those types of patterns that we actually need for tokens.
- In theory of compilation regular expressions are used to formalize the ***specification of tokens***
- Regular expressions are means for specifying regular languages

Strings and Languages

- An alphabet is any finite set of symbols. Typical examples of symbols are letters, digits, and punctuation.
  - The set {0, 1 } is the binary alphabet.
  - **ASCII** is an important example of an alphabet; it is used in many software systems.
  - **Unicode**, which includes approximately 100,000 characters from alphabets around the world, is another important example of an alphabet.

Specification of Tokens

- A **string** over an alphabet is a finite sequence of symbols drawn from that alphabet.
  - In language theory, the terms "***sentence***" and "***word***" are often used as synonyms for "***string***"
  - The ***length*** of a string ***s***, usually written **|*s*|**, is the number of occurrences of symbols in ***s***.
    - For example, **banana** is a string of length six.
  - The empty string, denoted ***ε***, is the string of length zero.
- A **language** is any countable set of strings over some fixed alphabet.
  - Abstract languages like ϕ, the empty set, or **{*ε*}** , the set containing only the empty string, are languages under this definition.
  - So too are the set of all syntactically well-formed java/c/c++ programs and the set of all grammatically correct English sentences, although the latter two languages are difficult to specify exactly.
- Note that the definition of "***language***" does not require that any "meaning" be ascribed to the strings in the language.

Specification of Tokens

Terms for Parts of String

- The following string-related terms are commonly used:
  1. A ***prefix*** of string ***S*** is any string obtained by removing zero or more symbols from the end of ***S***.
     - For example, ***ban***, ***banana***, and ***ε*** are prefixes of ***banana***.
  2. A ***suffix*** of string ***S*** is any string obtained by removing zero or more symbols from the beginning of ***S***.
     - For example, ***nana***, ***banana***, and ***ε*** are suffixes of ***banana***.
  3. A ***substring*** of ***S*** is obtained by deleting any prefix and any suffix from ***S***.
     - For instance, ***banana***, ***nan***, and ***ε*** are substrings of ***banana***.
  4. The ***proper prefixes***, ***suffixes***, and ***substrings*** of a string ***S*** are those, prefixes, suffixes, and substrings, respectively, of ***S*** that are not ***ε*** or not equal to ***S*** itself.
  5. A ***subsequence*** of ***S*** is any string formed by deleting zero or more not necessarily consecutive positions of ***S***.
     - For example, ***baan*** is a subsequence of banana.

Specification of Tokens

Operations on Languages

- The following string-related terms are commonly used:
- In lexical analysis, the most important operations on languages are ***union***, ***concatenation***, and ***closure***, which are defined formally below:

1. **Union (U)**:- is the familiar operation on sets.
   - Example Union of ***L*** and ***M***, $\boldsymbol{LUM = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}}$
2. **Concatenation**:- is all strings formed by taking a string from the first language and a string from the second language, in all possible ways, and concatenating them.
   - Example Concatenation of ***L*** and ***M***, $\boldsymbol{LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}}$
3. **Kleene closure**:- the kleene closure of a language ***L***, denoted by ***L****, is the set of strings you get by concatenating ***L*** zero, or more times.
   - Example Kleene closure of ***L***, $\boldsymbol{L^* = \bigcup_{i=0}^{\infty} L^i}$.
   - Note that:- $\boldsymbol{L^0}$, the "concatenation of ***L*** zero times," is defined to be $\{\varepsilon\}$, and inductively, L^i is $L^{i-1}L$.
4. **Positive closure**:- is the positive closure of a language ***L***, denoted by $\boldsymbol{L^+}$, is the same as the Kleene closure, but without the term $\boldsymbol{L^0}$. That is, ε will not be in $\boldsymbol{L^+}$ unless it is ***L*** in itself.
   - Example Positive Kleene closure of ***L***, $\boldsymbol{L^+ = \bigcup_{i=1}^{\infty} L^i}$.

Specification of Tokens

Operations on Languages

- Let ***L*** be the set of letters $\{A, B, \ldots, Z, a, b, \ldots, z\}$ and let ***D*** be the set of digits $\{0, 1, \ldots, 9\}$. We may think of ***L*** and ***D*** in two, essentially equivalent, ways.
  - One way is that ***L*** and ***D*** are, respectively, the alphabets of uppercase and lowercase letters and of digits.
  - The second way is that ***L*** and ***D*** are languages, all of whose strings happen to be of length one.
- Here are some other languages that can be constructed from languages ***L*** and ***D***, using the above operators:
  1. $\boldsymbol{L \cup D}$ is the set of letters and digits - strictly speaking the language with 62 strings of length one, each of which strings is either one letter or one digit.
  2. $\boldsymbol{LD}$ is the set of 520 strings of length two, each consisting of one letter followed by one digit.
  3. $\boldsymbol{L^4}$ is the set of all *4-letter* strings.
  4. $\boldsymbol{L^*}$ is the set of all strings of letters, including $\boldsymbol{\varepsilon}$, the empty string.
  5. $\boldsymbol{L(L \cup D)^*}$ is the set of all strings of letters and digits beginning with a letter.
  6. $\boldsymbol{D^+}$ is the set of all strings of one or more digits.

Specification of Tokens

Regular Expressions

- In theory of compilation regular expressions are used to formalize the specification of tokens.
- Regular expressions are means for specifying ***regular languages***
  - Example: `ID → letter_(letter|digit)*`
    `letter → A|B|…|Z|a|b|…|z|_`
    `digit → 0|1|2|…|9`
  - Each regular expression is a pattern specifying the form of strings
- The regular expressions are built recursively out of smaller regular expressions, using the following two rules:

  R1:- Ɛ is a regular expression, and $\boldsymbol{L}(\mathcal{E}) = \{\mathcal{E}\}$, that is, the language whose sole member is the empty string.

  R2:- If $\boldsymbol{a}$ is a symbol in ∑ then $\boldsymbol{a}$ is a regular expression, $\boldsymbol{L}(\boldsymbol{a}) = \{\boldsymbol{a}\}$, that is, the language with one string, of length one, with a in its one position.

  Note:- By convention, we use italics for symbols, and boldface for their corresponding regular expression.

  - Each regular expression $\boldsymbol{r}$ denotes a language $\boldsymbol{L}(\boldsymbol{r})$, which is also defined recursively from the languages denoted by $r's$ sub expressions.

Specification of Tokens

Regular Expressions

- There are four parts to the induction whereby larger regular expressions are built from smaller ones.
- Suppose $\boldsymbol{r}$ and $\boldsymbol{s}$ are regular expressions denoting languages $\boldsymbol{L}(\boldsymbol{r})$ and $\boldsymbol{L}(\boldsymbol{s})$, respectively.
  1. $(\boldsymbol{r})|(\boldsymbol{s})$ is a regular expression denoting the language $\boldsymbol{L}(\boldsymbol{r}) \cup \boldsymbol{L}(\boldsymbol{s})$.
  2. $(\boldsymbol{r})(\boldsymbol{s})$ is a regular expression denoting the language $\boldsymbol{L}(\boldsymbol{r})\boldsymbol{L}(\boldsymbol{s})$.
  3. $(\boldsymbol{r})^*$ is a regular expression denoting $(L(r))^*$.
  4. $(\boldsymbol{r})$ is a regular expression denoting $\boldsymbol{L}(\boldsymbol{r})$.
     - This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote.
- Regular expressions often contain unnecessary pairs of parentheses.
- We may drop certain pairs of parentheses if we adopt the conventions that:
  a) The unary operator * has highest precedence and is left associative.
  b) Concatenation has second highest precedence and is left associative.
  c) Union has lowest precedence and is left associative.

Example:- $(\boldsymbol{a})\,|((\boldsymbol{b})^*(\boldsymbol{c}))$ can be represented by $a|b^*c$.

– Both expressions denote the set of strings that are either a single a or are zero or more b' s followed by one c.

Specification of Tokens

Regular Expressions

- A language that can be defined by a regular expression is called a ***regular set***.
- If two regular expressions $\boldsymbol{r}$ and $\boldsymbol{s}$ denote the same regular set, we say they are equivalent and write $\boldsymbol{r = s}$.
  - For instance, $(\boldsymbol{a}|\boldsymbol{b}) = (\boldsymbol{b}|\boldsymbol{a})$.
- There are a number of algebraic laws for regular expressions; each law asserts that expressions of two different forms are equivalent.

LAW	DESCRIPTION
$\boldsymbol{r}\|\boldsymbol{s} = \boldsymbol{s}\|\boldsymbol{r}$	| is commutative
$\boldsymbol{r}\|(\boldsymbol{s}\|\boldsymbol{t}) = (\boldsymbol{r}\|\boldsymbol{s})\|\boldsymbol{t}$	| is associative
$\boldsymbol{r}(\boldsymbol{st}) = (\boldsymbol{rs})\boldsymbol{t}$	Concatenation is associative
$\boldsymbol{r}(\boldsymbol{s}\|\boldsymbol{t}) = \boldsymbol{rs}\|\boldsymbol{rt}$	Concatenation distributes over |
$(\boldsymbol{s}\|\boldsymbol{t})\boldsymbol{r} = \boldsymbol{sr}\|\boldsymbol{tr}$	
$\varepsilon\boldsymbol{r} = \boldsymbol{r}\varepsilon = \boldsymbol{r}$	ε is the identity for concatenation
$\mathbf{r^* = (r\|\varepsilon)^*}$	ε is guaranteed in a closure
$\mathbf{r^{**} = r^*}$	* is idempotent

Table: The algebraic laws that hold for arbitrary regular expressions **r, s,** and **t**.

Specification of Tokens

Regular Definitions

- If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form :

$d_1 \rightarrow r_1$
$d_2 \rightarrow r_2$
…
$d_n \rightarrow r_n$

Where

1. Each d_i is a new symbol, not in Σ and not the same as any other of the d's, and
2. Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$.

- **Examples**

a) Regular definition for C++ identifiers:

```
ID → letter(letter|digit)*
letter → A|B|…|Z|a|b|…|z|_
digit → 0|1|2|…|9
```

b) Regular definition for C++ statements

stmt → if expr then stmt
| if expr then stmt else stmt
| Ɛ
expr → term relop term
| term
term → id
| number

c) Regular Definition for unsigned numbers such as 5280, 0.0 1 234, 6. 336E4, or 1. 89E-4

digit →0|1|2|…|9
digits →digit digit*
optFrac→ .digts|Ɛ
optExp→ (E(+|-|Ɛ) digts|Ɛ
number → digits optFrac OptExp

Specification of Tokens

Extension of Regular Expressions

- Many extensions have been added to regular expressions to enhance their ability to specify string patterns.
- Here are few notational extensions:

1. **One or more instances(+)**:- is a ***unary postfix operator*** that represents the positive closure of a regular expression and its language. That is, if $\boldsymbol{r}$ is a regular expression, then $(r)^+$ denotes the language $(L(r))^+$.
   - » The operator + has the same precedence and associativity as the operator *.
   - » Two useful algebraic laws, $\boldsymbol{r^* = r + |\, \varepsilon}$ and $\boldsymbol{r^+ = rr^* = r^*r}$.
2. **Zero or one instance(?)**:- $\boldsymbol{r}?$ is equivalent to $\boldsymbol{r|\varepsilon}$, or put another way, $\boldsymbol{L(r?) = L(r) \cup \{\varepsilon\}}$.
   - » The **?** operator has the same precedence and associativity as * and +.
3. **Character classes**:- A regular expression $\boldsymbol{a_1|a_2|\cdots|a_n}$, where the a_i's are each symbols of the alphabet, can be replaced by the shorthand $\boldsymbol{[a_1a_2 \ldots a_n]}$.

   Example:

   $\boldsymbol{[abc]}$ is shorthand for $\boldsymbol{a|\, b\, |\, c}$, and

   $\boldsymbol{[a - z]}$ is shorthand for $\boldsymbol{a\, |b\, |c\, |\cdots\, |z}$.

Specification of Tokens

Extension of Regular Expressions

- Using the above short hands we can rewrite the regular expression for examples a) and c) in slide number 32 as follows:

  Examples

  a) Regular definitions for C++ identifiers

  ```
  ID → letter(letter|digit)*
  letter →[A-Za-z_]
  digit →[0-9]
  ```

  c) Regular Definition for unsigned numbers such as 5280 , 0.0 1 234, 6. 336E4, or 1. 89E-4

  digit →[0-9]
  digits →digit^{+}
  number → digits (. digits)? (E [+ -]? digits)?

Specification of Tokens

Extension of Regular Expressions

Exercises - Assignments

1. Consult the language reference manuals for C++ and java programming languages to determine
   A. The sets of characters that form the input alphabet (excluding those that may only appear in character strings or comments) ,
   B. The lexical form of numerical constants, and
   C. The lexical form of identifiers.
2. Describe the languages denoted by the following regular expressions:
   a) $a(a|b)^*a$ b) $((\varepsilon|a)\ b^*)^*$ c) $(a|b)^*a(a|b)(a|b)$
   d) $a^*ba^*ba^*ba^*$ e) $(aa|bb)^*\big((ab|ba)(aa|bb)^*\ (ab|ba)(aa|bb)^*\big)^*$
3. Write regular definitions for the following languages:
   a) All strings of lowercase letters that contain the five vowels in order.
   b) All strings of lowercase letters in which the letters are in ascending lexicographic order.
   c) All strings of binary digits with no repeated digits.
   d) All strings of binary digits with at most one repeated digit.
   e) All strings of a's and b's where every a is preceded by b.
   f) All strings of a's and b's that contain substring $abab$.

Recognition of Tokens

- Regular definitions, a mechanism based on regular expressions are very popular for specification of tokens.
  - Has been implemented in the lexical analyzer generator tool, ***LEX***.
  - In our lab session, we will see token specification using ***LEX***.
- Transition diagrams, a variant of finite state automata, are used to implement regular definitions and to ***recognize tokens***.
  - Transition diagrams are usually used to model a Lexical Analyzer before translating them to programs by hand.
  - LEX automatically generates optimized FSA from regular definitions
  - We studied FSA and their generation from regular expressions, then we will see the relation between transition diagrams and LEX.

Recognition of Tokens

Transition Diagrams

- Transition diagrams are generalized DFAs with the following differences
  - Edges may be labelled by ***a symbol***, ***a set of symbols***, or ***a regular definition***.
  - Some accepting states may be indicated as ***retracting states***, indicating that the lexeme does not include the symbol that brought us to the accepting state.
  - Each accepting state has an action attached to it, which is executed when that state is reached. Typically, such an action returns a token and its attribute value.
- Transition diagrams are not meant for machine translation but only for manual translation.

Recognition of Tokens

1. Starting point is the language grammar to understand the tokens:

stmt → if expr then stmt
| if expr then stmt else stmt
| ε
expr → term relop term
| term
term → id
| number

2. The next step is to formalize the patterns:

digit → [0-9]
Digits → digit+
number → digit(.digits)? (E[+-]? Digit)?
letter → [A-Za-z_]
id → letter (letter|digit)*
If → if
Then → then
Else → else
Relop → < | > | <= | >= | = | <>

3. We also need to handle whitespaces:

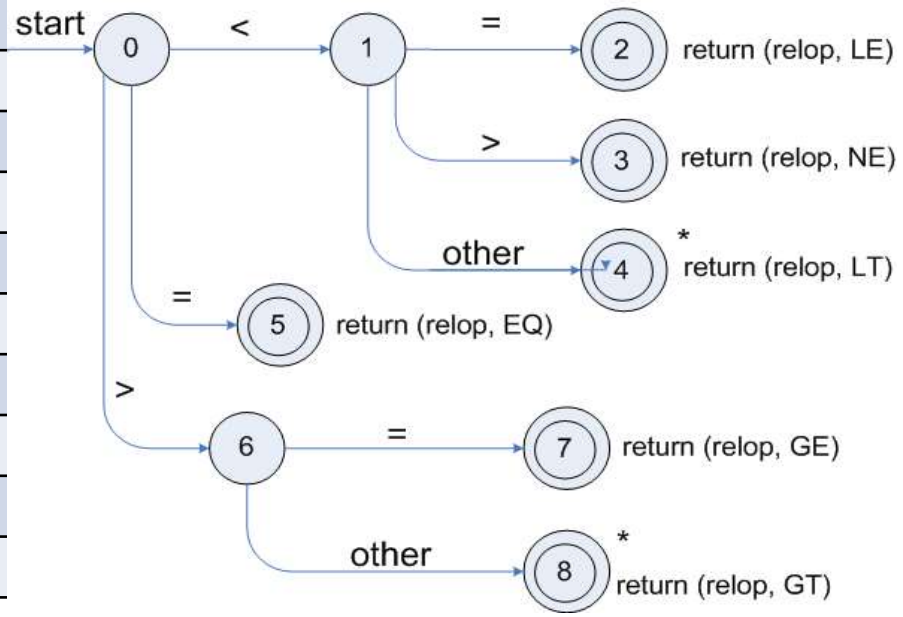
ws → (blank | tab | newline)+

Recognitions of Tokens

- To understand the recognition of tokens, consider the regular expression of tokens, together with attribute values

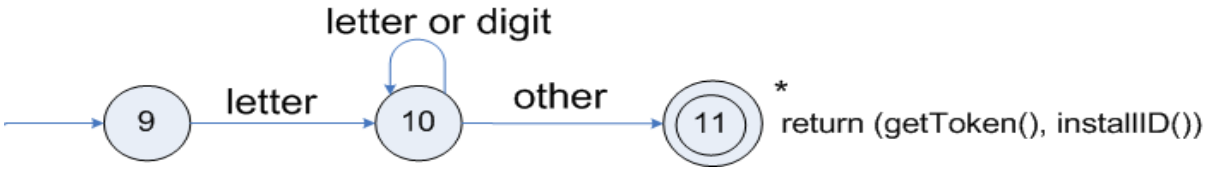
Lexemes	Token Names	Attribute Value
Any ws	-	-
if	if	-
then	then	-
else	else	-
Any id	id	Pointer to table entry
Any number	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

- Transition diagram for **relop**

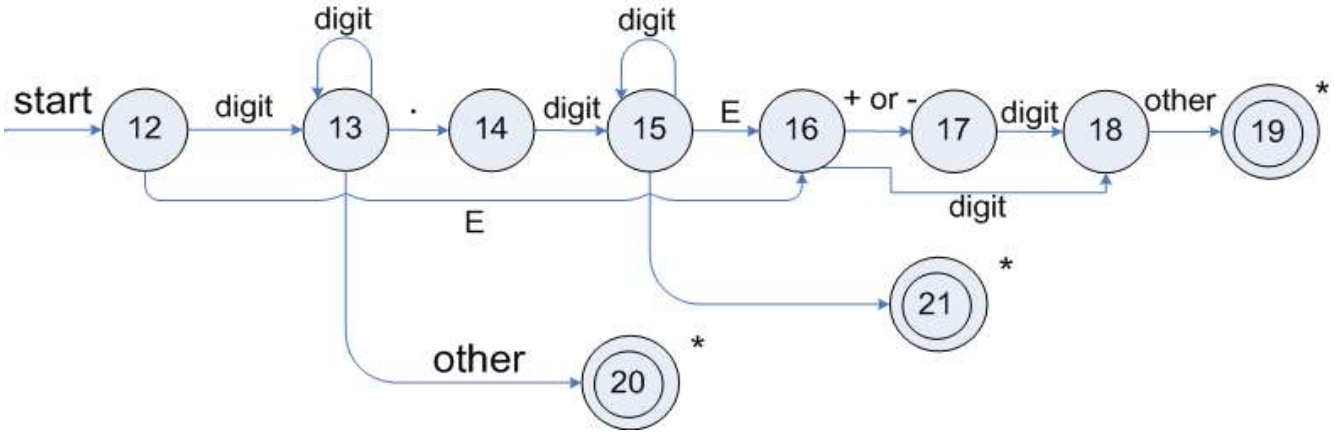


Recognitions of Tokens

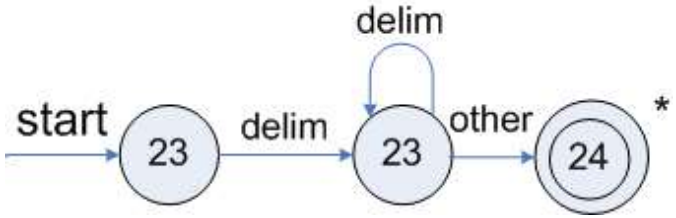
- Transition diagram Transition diagram for ***reserved words*** and ***identifiers***:



- Transition diagram for ***unsigned numbers***:



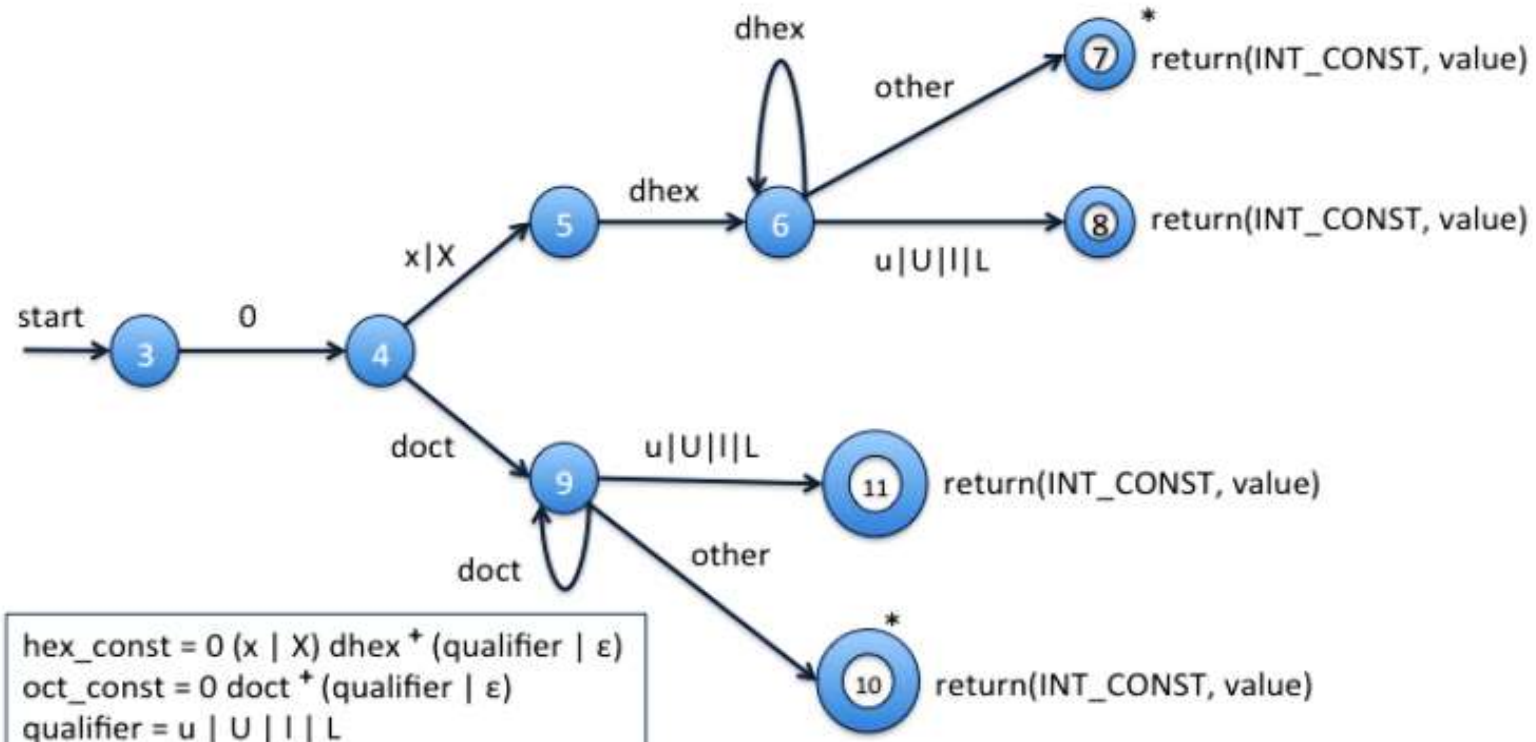
- Transition diagram for ***white spaces*** where ***delim*** represents one or more whitespace characters:



- '*******' indicates the ***retraction state***.
- ***`getToken()`*** searches a table to check if the ***name*** is a reserved word and returns its integer code if so.

Recognitions of Tokens

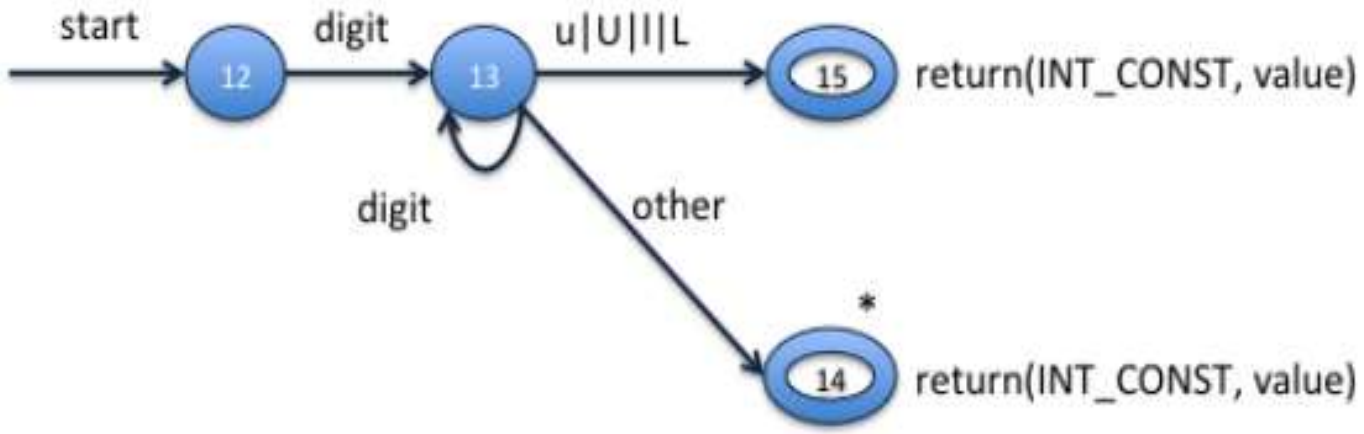
- In a similar way, transition diagrams for other program constructs can be designed as follows:
- For **hexadecimal** and **octal constants**:



Recognitions of Tokens

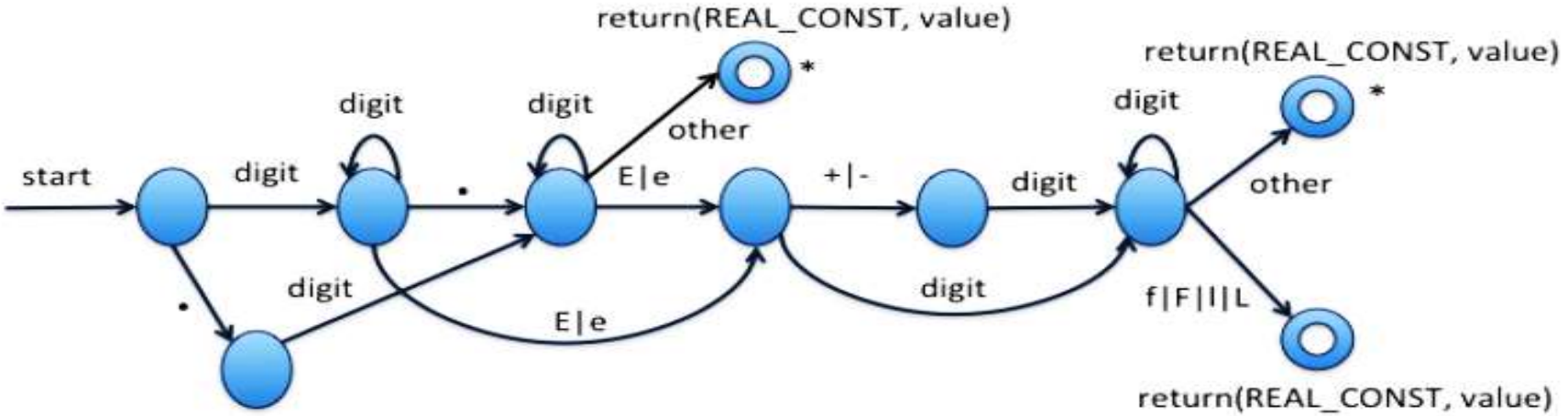
- For **Integer constants**:

int_const = digit $^{+}$ (qualifier | ε)
qualifier = u | U | l | L
digit = [0-9]



Recognitions of Tokens

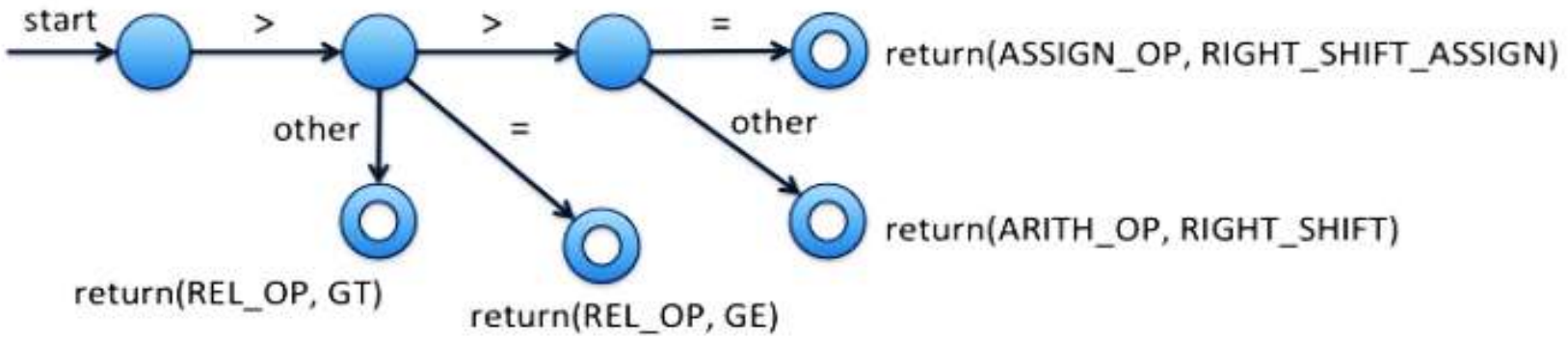
- For **Real constants**:

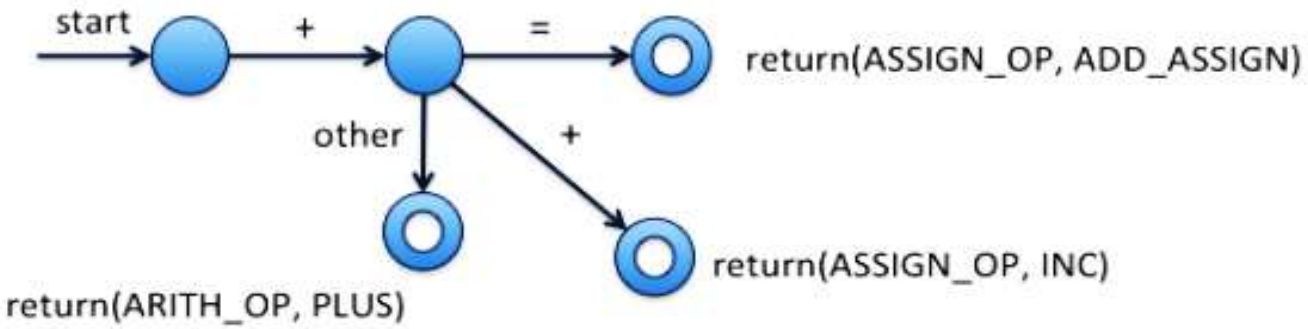



```
real_const = (digit + exponent (qualifier | ε)) |
             (digit* "." digit + (exponent | ε) (qualifier | ε)) |
             (digit + "." digit* (exponent | ε) (qualifier | ε))
exponent = (E|e)(+|-|ε) digit +
qualifier = f | F | l | L
digit = [0-9]
```

Recognitions of Tokens

- For **a few operators**:





Combining TDs to form Lexical Analyzer

- Different transition diagrams must be combined appropriately to yield a Lexical Analyzer.
  - Combining TDs is not trivial
  - It is possible to try different transition diagrams one after another
  - For example, TDs for reserved words, constants, identifiers, and operators could be tried in that order
  - However, this does not use the "***longest match***" characteristic (the ***next*** would be an identifier, and not reserved word then followed by identifier ***ext***)
  - To find the longest match, all TDs must be tried and the longest match must be used
  - Using LEX to generate a lexical analyzer makes it easy for the compiler writer

Lexical Analyzer Implementation for TDs

- The following code can be constructed by combining the transition diagrams:

```
TOKEN gettoken() {
   TOKEN mytoken; char c;
   while(1) { switch (state) {
     /* recognize reserved words and identifiers */
       case 0: c = nextchar(); if (letter(c))
               state = 1; else state = failure();
               break;
       case 1: c = nextchar();
               if (letter(c) || digit(c))
               state = 1; else state = 2; break;
       case 2: retract(1);
               mytoken.token = search_token();
               if (mytoken.token == IDENTIFIER)
               mytoken.value = get_id_string();
               return(mytoken);
```

Lexical Analyzer Implementation for TDs

```
/* recognize hexa and octal constants */
  case 3: c = nextchar();
          if (c == '0') state = 4; break;
          else state = failure();
  case 4: c = nextchar();
          if ((c == 'x') || (c == 'X'))
          state = 5; else if (digitoct(c))
          state = 9; else state = failure();
          break;
  case 5: c = nextchar(); if (digithex(c))
          state = 6; else state = failure();
          break;
```

Lexical Analyzer Implementation for TDs

```
    case 6: c = nextchar(); if (digithex(c))
            state = 6; else if ((c == 'u')||
            (c == 'U')||(c == 'l')||
            (c == 'L')) state = 8;
            else state = 7; break;
    case 7: retract(1);
/* fall through to case 8, to save coding */
    case 8: mytoken.token = INT_CONST;
            mytoken.value = eval_hex_num();
            return(mytoken);
    case 9: c = nextchar(); if (digitoct(c))
            state = 9; else if ((c == 'u')||
            (c == 'U')||(c == 'l')||(c == 'L'))
            state = 11; else state = 10; break;
```

Lexical Analyzer Implementation for TDs

```
    case 10: retract(1);
/* fall through to case 11, to save coding */
    case 11: mytoken.token = INT_CONST;
             mytoken.value = eval_oct_num();
             return(mytoken);
```

Lexical Analyzer Implementation for TDs

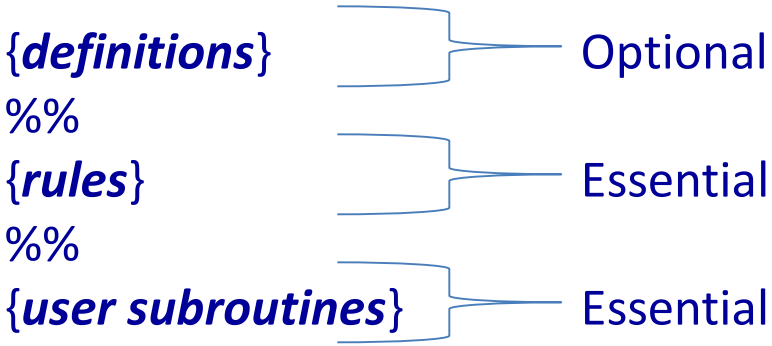
```
    /* recognize integer constants */
        case 12: c = nextchar(); if (digit(c))
                 state = 13; else state = failure();
        case 13: c = nextchar(); if (digit(c))
                 state = 13;else if ((c == 'u')||
                 (c == 'U')||(c == 'l')||(c == 'L'))
                 state = 15; else state = 14; break;
        case 14: retract(1);
    /* fall through to case 15, to save coding */
        case 15: mytoken.token = INT_CONST;
                 mytoken.value = eval_int_num();
                 return(mytoken);
        default: recover();
        }
    }
}
```

Introduction to *Lex*

A lexical analyzer generator tool

Introduction to *Lex*

- ***LEX*** – is ***pattern-action language*** / ***pattern directed programming*** used for specifying lexical analyzers.
- ***LEX*** has a language for describing regular expressions
  - It generates a pattern matcher for the regular expression specifications provided to it as input.
  - **General structure of a LEX program is:**



  - **Commands to create an LA**

    `lex ex.l` – creates a C-program ***`lex.yy.c`***

    `gcc -o ex.o lex.yy.c` – produces ***`ex.o`***

    `ex.o` is a lexical analyzer, that carves tokens from its input

Introduction to *Lex*

Lex Example 1: LEX specification for the recognizing Capital Letter

```
/* no definition sectin */
%%
[A-Z]+    { ECHO; printf("\n"); }
.|\n      ;
%%
yywrap(){}
main(){
    yylex();
}
```

Rule section

Subroutine section

```
/* Input */
wewevWEUFWIGhHkkH
sdcwehSDWEhTkFLksewT
```

```
/* Output */
WEUFWIG
H
H
SDWE
T
FL
T
```

Introduction to *Lex*

Lex – Definition Section

- contains definitions and included code.
  - Definitions are like macros and have the following form:

```
name            translation
digit           [0-9]
number          {digit}{digit}*
```

  - Included code is all code included between %{ and %}:

```
%{
        float number; int count=0;
%}
```

Introduction to *Lex*

Lex – Rule Section

- Contains ***patterns*** and ***C-code***
  - A line starting with white space or material enclosed in %{ and %} is a C-code.
  - A line starting with anything else is a pattern line.
  - Pattern lines contain a pattern followed by some white space and C-code:

    ```
    { pattern }                    { action (C – code) }
    ```

  - C-code lines are copied verbatim to the generated C-file (***lex.yy.c***).
- ***Patterns*** are translated into ***NFA*** which are then converted into ***DFA***, optimized, and stored in the form of a state table and a driver routine.
- The ***action*** associated with a pattern is executed when the ***DFA*** recognizes a string corresponding to that pattern and reaches a final state.

Introduction to *Lex*

Lex – *Metacharacters* and *Matches*

- Regular expressions in ***lex*** are composed of ***metacharacters***.

Pattern Matching Primitives

Metacharacter	*Matches*
`.`	any character except newline
`\n`	newline
`*`	zero or more copies of preceding expression
`+`	one or more copies of preceding expression
`?`	zero or one copy of preceeding expression
`^`	beginning of line
`$`	end of line
`a|b`	a or b
`(ab)+`	one or more copies of ab (grouping)
`"a+b"`	literal “a+b” (C escapes still work)
`[]`	character class

Introduction to *Lex*

Lex – *Metacharacters* and *Matches*

- Pattern matching example:

Expression	*Matches*
`abc`	`abc`
`abc*`	`ab, abc, abcc, abccc, …`
`abc+`	`abc, abcc, abccc, …`
`a(bc)+`	`abc, abcbc, abcbcbc, …`
`a(bc)?`	`a, abc`
`[abc]`	`a, b, c`
`[a-z]`	any letter, `a` through `z`
`[a\-z]`	`a, -, z`
`[-az]`	`-, a, z`
`[A-Za-z0-9]+`	one or more alphanumeric characters
`[ \t\n]+`	whitespace
`[^ab]`	anything except: `a, b`
`[a^b]`	`a, ^, b`
`[a|b]`	`a, |, b`
`a|b`	`a or b`

Lex – Strings and Operators

- **Examples of strings**: *integer a57d hello*

- **Operators**: `" \ [] ^ - ? . * + | () $ {} % <>`
  - `\` (backslash) can be used as an escape character as in C
- **Character classes**: enclosed in **[** and **]**
  - Only `\, -,` and `^` are special inside `[ ]`.
  - All other operators are irrelevant inside `[ ]`.
- **Examples**:

`[-+][0-9]+`	--->	`(-|+)(0|1|2|3|4|5|6|7|8|9)+`
`[a-d][0-4][A-C]`	--->	`a|b|c|d|0|1|2|3|4|A|B|C`
`[^abc]`	--->	all char except `a`, `b`, or `c`, including special and control char.
`[+\-][0-5]+`	--->	`(+|-)(0|1|2|3|4|5)+`
`[^a-zA-Z]`	--->	all char which are not letters

Lex – Strings and Operators

- **The `.` operator**: matches any character except newline.
- **The `?` operator**: used to implement option **`ab?c`** stands for **`a(b|ε)c`**
- **Repetition, alternation, and grouping**:
  Example: `(ab|cd+)?(ef)*` ---> `(ab|c(d)+ | ε)(ef)*`
- **Context sensitivity**: **`/`**, **`^`**, **`$`**, are context-sensitive operators
  - **`^`** : If the first char of an expression is **`^`**, then that expression is matched only at the beginning of a line. Holds only outside [] operator.
  - **`$`** : If the last char of an expression is **`$`**, then that expression is matched only at the end of a line.
  - **`/`** : ***Look ahead operator***, indicates trailing context.

  Examples:

  - **`^ab`** **--->** line beginning with **`ab`**
  - **`ab$`** **--->** line ending with ab (same as **`ab/\n`**)
  - **`DO/({letter}|{digit})* = ({letter}|{digit})*`**

Lex – Actions

- ***Default action*** is to copy input to output those characters which are unmatched.
- We need to provide patterns to catch characters.
  - **`yytext`**: contains the text matched against a pattern
    - copying **`yytext`** can be done by the action **`ECHO`**
  - **`yyleng`**: provides the number of characters matched.
- **LEX** always tries the rules in the order written down and the longest match is preferred
  - **Example**:

```
%%
integer       action1;
[a-z]+        action2;
%%
```

  - The input ***`integers`*** will match the second pattern & **`action2`** will be performed.

Lex – Actions

Lex Predefined Variables

name	*function*
`int yylex(void)`	call to invoke lexer, returns token
`char *yytext`	pointer to matched string
`yyleng`	length of matched string
`yylval`	value associated with token
`int yywrap(void)`	wrapup, return 1 if done, 0 if not done
`FILE *yyout`	output file
`FILE *yyin`	input file
`INITIAL`	initial start condition
`BEGIN condition`	switch start condition
`ECHO`	write matched string

Lex – Examples

- **Example 2**:number of lines

```
%%
^[ ]*\n
\n          {ECHO; yylineno++;}
.*          {printf("%d\t%s",yylineno,yytext);}
%%
yywrap(){}
main(){
  yylineno = 1;
  yylex();
}
```

Lex – Examples

- **Example 3:** declaration statements

```
%{
     FILE *declfile;
%}
Blanks      [ \t]*
Letter      [a-z]
Digit       [0-9]
id          ({letter}|_)({letter}|{digit}|_)*
number      {digit}+
arraydeclpart {id}"["{number}"]"
declpart    ({arraydeclpart}|{id})
decllist  ({declpart}{blanks}","{blanks})*{blanks}{declpart}{blanks}
declaration (("int")|("float")){blanks}{decllist}{blanks};
%%
{declaration}  fprintf(declfile,"%s\n",yytext);
%%
yywrap(){ fclose(declfile); }
main(){ declfile = fopen("declfile","w"); yylex(); }
```

Lex – Examples

- **Example 4 : Identifier, Reserved words, and constants**

```
%{
    int hex = 0; int oct = 0; int regular =0;
%}
letter          [a-zA-Z_]
digit           [0-9]
digits          {digit}+
digit_oct       [0-7]
digit_hex       [0-9A-F]
int_qualifier   [uUlL]
blanks          [ \t]+
identifier      {letter}({letter}|{digit})*
integer         {digits}{int_qualifier}?
hex_const       0[xX]{digit_hex}+{int_qualifier}?
oct_const       0{digit_oct}+{int_qualifier}?
```

Lex – Examples

- **Example 4**: Identifier, Reserved words, and constants

```
%%
if          {printf("reserved word:%s\n",yytext);}
else        {printf("reserved word:%s\n",yytext);}
while       {printf("reserved word:%s\n",yytext);}
switch      {printf("reserved word:%s\n",yytext);}
{identifier}      {printf("identifier :%s\n",yytext);}
{hex_const}       {sscanf(yytext,"%i",&hex); printf("hex
                   constant:%s = %i\n",yytext,hex);}
{oct_const}       {sscanf(yytext,"%i",&oct); printf("oct
                   constant:%s = %i\n",yytext,oct);}
{integer}         {sscanf(yytext,"%i",&regular); printf(
                  "integer :%s = %i\n",yytext, regular);}
.|\n              ;
%%
yywrap(){}
int main(){yylex();}
```

Lex – Examples

- **Example 5: Floats in c**

```
digits      [0-9]+
exp         ([Ee](\+|\-)?{digits})
blanks      [ \t\n]+
float_qual          [fFlL]
%%
{digits}{exp}{float_qual}?/{blanks}
            {printf("float no fraction: %s\n",yytext);}
[0-9]*\.{digits}{exp}?{float_qual}?/{blanks}
            {printf("float with optionalinteger part:
             %s\n",yytext);}
{digits}\.[0-9]*{exp}?{float_qual}?/{blanks}
           printf("float with optional fraction:
           %s\n",yytext);}
.|\n           ;
%%
yywrap(){}
int main(){yylex();}
```

End of CH2!

- **Thank You!**
  - **Any Questions…?**